

LINEAR REGRESSION

Complete Assignment Answers – Questions 1 to 10

Question 1: What is Simple Linear Regression?

Simple Linear Regression is a supervised Machine Learning/statistical technique used to model the relationship between **one independent variable (X)** and **one dependent variable (Y)**.

It assumes that the relationship between X and Y can be represented by a straight line.

Formula

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- (Y) = dependent/target variable
- (X) = independent/input variable
- (β_0) = intercept
- (β_1) = slope/coefficient
- (ϵ) = error term

The predicted value is commonly written as:

$$Y = \beta_0 + \beta_1 X$$

Example

Suppose we want to predict a student's marks based on study hours.

Study Hours $\rightarrow$ Marks

1 $\rightarrow$ 40

2 $\rightarrow$ 50

3 $\rightarrow$ 60

Here:

- Study Hours = independent variable
- Marks = dependent variable

Applications

- Predicting house prices
- Predicting sales
- Predicting salary
- Predicting demand

- Predicting marks

Easy Definition

Simple Linear Regression predicts a dependent variable using one independent variable through a straight-line relationship.

Question 2: What Are the Key Assumptions of Simple Linear Regression?

Simple Linear Regression relies on several important assumptions.

1. Linearity

There should be a reasonably linear relationship between X and Y.

`X ↑ → Y tends to change in a straight-line pattern`

2. Independence

Observations/errors should be independent of each other.

One observation should not systematically depend on another.

3. Homoscedasticity

The variance of the residuals should remain approximately constant across predicted values.

If residual spread changes significantly, this is called **heteroscedasticity**.

4. Normality of Residuals

For classical statistical inference, residuals are generally assumed to be approximately normally distributed.

This assumption is especially important for reliable confidence intervals and hypothesis tests.

5. No Significant Outliers/Influential Points

Extreme observations should not disproportionately control the regression line.

6. Correct Model Specification

The model should include appropriate predictors and functional form.

Summary Table

Assumption	Meaning
Linearity	X and Y have a linear relationship
Independence	Observations/errors are independent
Homoscedasticity	Constant residual variance
Normality	Residuals are approximately normal
No influential outliers	Extreme points should not dominate
Correct specification	Appropriate variables/form are included

Easy Definition

Good linear regression requires linearity, independence, constant error variance, suitable residual behavior, and no problematic influential observations.

Question 3: What Is Heteroscedasticity, and Why Is It Important to Address in Regression Models?

Heteroscedasticity occurs when the variance of regression errors/residuals is **not constant** across the range of fitted values or an explanatory variable.

In simple terms, the residuals may form a pattern where their spread increases or decreases.

Example

Suppose we predict house prices.

For inexpensive houses:

Small prediction errors

For expensive houses:

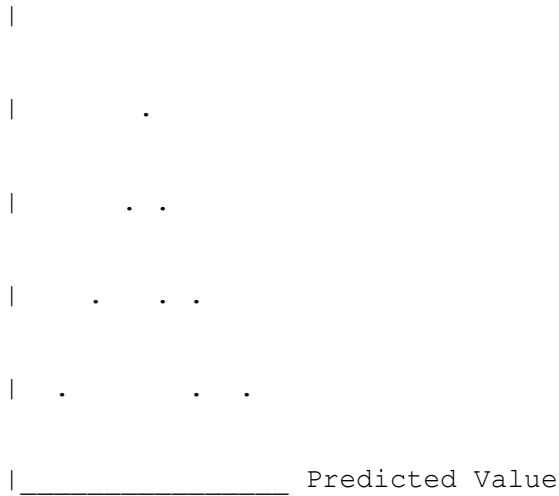
Large prediction errors

The changing spread of errors indicates heteroscedasticity.

Visual Representation

A residual plot may look like:

Residual



This widening pattern is often called a **funnel shape**.

Why Is It Important?

Heteroscedasticity can:

- Make standard errors unreliable.
- Affect confidence intervals.
- Affect hypothesis-test results.
- Make coefficient significance tests less trustworthy.
- Indicate that the model does not capture the error structure well.

The ordinary least-squares coefficient estimates can still be unbiased under some conditions, but their usual standard errors may be incorrect.

How Can It Be Detected?

Residual plots can be used.

```
plt.scatter(y_pred, residuals)
```

Formal tests such as the **Breusch-Pagan test** can also be used.

Ways to Handle It

1. Transform the target, e.g. log transformation.
2. Transform relevant predictors.
3. Use weighted least squares.
4. Use heteroscedasticity-robust standard errors.
5. Investigate missing variables or incorrect model specification.

Easy Definition

Heteroscedasticity = Unequal variance of regression errors.

Question 4: What is Multiple Linear Regression?

Multiple Linear Regression is a regression technique used to predict a dependent variable using **two or more independent variables**.

Formula

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \epsilon$$

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k + \epsilon$$

Where:

- (Y) = dependent variable
- $(X_1, X_2, \dots, X_k)$ = independent variables
- (β_0) = intercept
- $(\beta_1, \beta_2, \dots, \beta_k)$ = coefficients
- (ϵ) = error

Example

House price can depend on:

- Area
- Number of rooms
- Age of house
- Location

Therefore:

$$\text{Price} = \beta_0 + \beta_1 \text{Area} + \dots + \beta_k \text{Location}$$

$$\text{Area} + \beta_2$$

$$\text{Rooms} + \beta_3$$

$$\text{Age} + \epsilon$$

Python Example

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
X = [
    [1000, 2],
    [1500, 3],
    [2000, 4]
]
```

```
y = [200000, 300000, 400000]
```

```
model.fit(X, y)
```

```
print("Coefficients:", model.coef_)

print("Intercept:", model.intercept_)
```

Difference from Simple Linear Regression

Simple Linear Regression	Multiple Linear Regression
One predictor	Two or more predictors
$(Y = \beta_0 + \beta_1 X)$	$(Y = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)$
Simpler	More complex
One feature	Multiple features

Easy Definition

Multiple Linear Regression predicts one target using two or more independent variables.

Question 5: What Is Polynomial Regression, and How Does It Differ from Linear Regression?

Polynomial Regression is used when the relationship between the independent and dependent variables is better represented by a curved relationship rather than a straight line.

Formula

For a second-degree polynomial:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

For a third-degree polynomial:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 +$$

$$\beta_3 X_3 + \epsilon$$

Example

Linear Regression:

Y

| /

| /

| /

| /

| _____ X

Polynomial Regression:

Y

| .

| .

| .

| .

| . _____ x

Important Point

Polynomial Regression is **linear in its coefficients**, even though it models a nonlinear relationship in X.

For example:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2$$

is still fitted using linear regression after creating polynomial features.

Python Example

```
import numpy as np
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.linear_model import LinearRegression
```

```
X = np.array([[1], [2], [3], [4], [5]])
```

```
y = np.array([2.2, 4.8, 7.5, 11.2, 14.7])
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

```
model = LinearRegression()
```

```
model.fit(X_poly, y)
```

```
print("Coefficients:", model.coef_)
```

```
print("Intercept:", model.intercept_)
```

Difference

Linear Regression	Polynomial Regression
Straight-line relationship	Curved relationship
Uses X	Uses X, X ² , X ³ , etc.
Simpler	More flexible
Less risk of overfitting at low complexity	High degree can overfit

Easy Definition

Polynomial Regression extends linear regression by adding powers of the features to model curved relationships.

Question 6: Implement Simple Linear Regression

Given Data

X = [1, 2, 3, 4, 5]

Y = [2.1, 4.3, 6.1, 7.9, 10.2]

Python Program

```
import numpy as np

import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
# Input data
```

```
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
```

```
Y = np.array([2.1, 4.3, 6.1, 7.9, 10.2])
```

```
# Create model
```

```
model = LinearRegression()
```

```
# Train model
```

```
model.fit(X, Y)
```

```
# Predictions
```

```
Y_pred = model.predict(X)
```

```
# Display model parameters
```

```
print("Slope:", model.coef_[0])
```

```
print("Intercept:", model.intercept_)
```



```
# Predict a new value

new_prediction = model.predict([[6]])

print("Prediction for X = 6:", new_prediction[0])


# Plot actual data

plt.figure(figsize=(8, 5))


plt.scatter(X, Y, label="Actual Data")

plt.plot(X, Y_pred, linewidth=2, label="Regression Line")


plt.title("Simple Linear Regression")

plt.xlabel("X")

plt.ylabel("Y")

plt.legend()


plt.show()
```

Expected Model

The fitted line will be approximately:

$$\hat{Y} \approx 0.08 + 2.00X$$

The exact coefficient values can be obtained by running the code.

Interpretation

The slope is approximately **2**, meaning that an increase of one unit in X is associated with an increase of about 2 units in predicted Y.

Question 7: Fit a Multiple Linear Regression Model

Given Data

```
Area = [1200, 1500, 1800, 2000]
```

```
Rooms = [2, 3, 3, 4]
```

```
Price = [250000, 300000, 320000, 370000]
```

Python Program

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

```
# Features
```

```
Area = np.array([1200, 1500, 1800, 2000])
```

```
Rooms = np.array([2, 3, 3, 4])
```

```
# Target
```

```
Price = np.array([250000, 300000, 320000, 370000])
```

```
# Create feature matrix
```

```
X = np.column_stack((Area, Rooms))
```

```
# Create model
```

```
model = LinearRegression()
```

```
# Train model
```

```
model.fit(X, Price)
```

```
# Display coefficients
```

```
print("Area Coefficient:", model.coef_[0])

print("Rooms Coefficient:", model.coef_[1])

print("Intercept:", model.intercept_)


# Predictions on training data

predictions = model.predict(X)


print("\nPredicted Prices:")

print(predictions)


# Predict price for a new house

new_house = [[2200, 4]]

new_prediction = model.predict(new_house)

print("\nPredicted Price for Area=2200 and Rooms=4:")

print(new_prediction[0])
```

Model Formula

The model follows:

$$\begin{aligned} \text{Price} = & \beta_0 \\ & + \beta_1 \\ & \text{Area} + \beta_2 \\ & \text{Rooms} \end{aligned}$$

Interpretation

- `Area Coefficient` represents the change in predicted price associated with one unit increase in area, **holding rooms constant**.
 - `Rooms Coefficient` represents the change in predicted price associated with one additional room, **holding area constant**.
-

Question 8: Implement Polynomial Regression

Given Data

`X = [1, 2, 3, 4, 5]`

`Y = [2.2, 4.8, 7.5, 11.2, 14.7]`

Python Program

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression


# Data

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)

Y = np.array([2.2, 4.8, 7.5, 11.2, 14.7])


# Create polynomial features

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)


# Create model

model = LinearRegression()
```

```
# Train model

model.fit(X_poly, Y)


# Predictions

Y_pred = model.predict(X_poly)


# Display model parameters

print("Intercept:", model.intercept_)

print("Coefficients:", model.coef_)


# Plot

plt.figure(figsize=(8, 5))


plt.scatter(X, Y, label="Actual Data")

plt.plot(X, Y_pred, linewidth=2, label="Polynomial Regression")


plt.title("Polynomial Regression")

plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.legend()
```

```
plt.show()
```

Explanation

```
PolynomialFeatures(degree=2)
```

creates features such as:

1, X, X²

The model then learns:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2$$

$$X^2$$

Why Degree 2?

The supplied data shows some curvature, so a quadratic model is a reasonable demonstration.

Question 9: Create a Residuals Plot for a Regression Model

Given Data

```
X = [10, 20, 30, 40, 50]
```

```
Y = [15, 35, 40, 50, 65]
```

Python Program

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
# Data

X = np.array([10, 20, 30, 40, 50]).reshape(-1, 1)

Y = np.array([15, 35, 40, 50, 65])


# Create and train model

model = LinearRegression()

model.fit(X, Y)


# Predictions

Y_pred = model.predict(X)


# Calculate residuals

residuals = Y - Y_pred


print("Actual Values:", Y)

print("Predicted Values:", Y_pred)

print("Residuals:", residuals)
```

```
# Residual plot

plt.figure(figsize=(8, 5))

plt.scatter(Y_pred, residuals)

plt.axhline(

    y=0,

    linestyle="--",

    linewidth=2

)

plt.title("Residuals Plot")

plt.xlabel("Predicted Values")

plt.ylabel("Residuals")

plt.show()
```

What Is a Residual?

A residual is the difference between the actual value and predicted value.

$$\text{Residual} = Y - \hat{Y}$$

For example, if:

Actual = 50

Predicted = 47

then:

$$\text{Residual} = 50 - 47 = 3$$

How to Interpret the Residual Plot

A good residual plot generally shows:

- Points randomly scattered around zero.
- No obvious curve.
- No obvious funnel/widening pattern.
- Approximately constant spread.

If We See a Pattern

A pattern may indicate:

- Non-linearity
- Heteroscedasticity
- Missing variables
- Incorrect model specification

Easy Definition

Residual = Actual value – Predicted value.

Question 10: Real Estate Case – Addressing Heteroscedasticity and Multicollinearity

Suppose we are working for a real estate company and want to predict house prices using:

- Area
- Number of rooms
- Location

We detect:

1. Heteroscedasticity
2. Multicollinearity

We should address both before finalizing the model.

Part A: Handling Heteroscedasticity

Step 1: Examine Residuals

First, fit the model and create a residual plot.

```
residuals = y_test - y_pred
```

```
plt.scatter(y_pred, residuals)
```

```
plt.axhline(0, linestyle="--")
```

```
plt.xlabel("Predicted Price")

plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.show()
```

If the residual spread increases with predicted price, heteroscedasticity may be present.

Step 2: Transform the Target

House prices are often positively skewed. A log transformation can sometimes stabilize variance.

```
y_log = np.log1p(y)
```

After prediction, convert back:

```
predicted_price = np.expml(predicted_log_price)
```

Step 3: Use Robust Standard Errors

If inference is the main goal, heteroscedasticity-robust standard errors can provide more reliable uncertainty estimates.

Step 4: Weighted Least Squares

If the variance structure can be modeled, weighted least squares can give less influence to observations with higher error variance.

Part B: Handling Multicollinearity

Multicollinearity occurs when two or more independent variables are strongly related to each other.

Example

Suppose:

Area ↔ Number of Rooms

Larger houses often have more rooms, so these predictors may contain overlapping information.

Step 1: Check Correlation

```
correlation = df[["Area", "Rooms"]].corr()
```

```
print(correlation)
```

Step 2: Calculate VIF

Variance Inflation Factor (VIF) is commonly used to diagnose multicollinearity.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
X = df[["Area", "Rooms"]]

vif = pd.DataFrame()

vif["Feature"] = X.columns

vif["VIF"] = [

    variance_inflation_factor(X.values, i)

    for i in range(X.shape[1])

]

print(vif)
```

A high VIF indicates potentially problematic multicollinearity. There is no single universal cutoff, although values around 5 or 10 are often used as warning thresholds depending on context.

Step 3: Remove Redundant Variables

If two variables provide very similar information, one may be removed based on domain knowledge and model goals.

For example:

Area

Rooms

If they are highly redundant, we may retain the more useful variable.

Step 4: Combine Features

We may create a more meaningful feature, such as:

$$\text{AreaPerRoom} = \frac{\text{Area}}{\text{Rooms}}$$

```
df["Area_Per_Room"] = df["Area"] / df["Rooms"]
```

Step 5: Use Regularization

Regularized regression can reduce the effect of multicollinearity.

Ridge Regression

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

Lasso Regression

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

Ridge is often particularly useful when predictors are highly correlated.

Part C: Location Feature Engineering

Location is categorical, so it should be encoded appropriately.

One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder
```

```
encoder = OneHotEncoder(handle_unknown="ignore")
```

```
location_encoded = encoder.fit_transform(
```

```
    df[["location"]]
```

)

For a complete ML workflow, use a `ColumnTransformer/Pipeline` so preprocessing is learned only from the training data.

Part D: Build a Robust Model

A practical workflow would be:

Raw Data

↓

Data Cleaning

↓

EDA

↓

Missing Value Handling

↓

Categorical Encoding

↓

Train-Test Split

↓

Check Multicollinearity



Feature Selection / Regularization



Fit Regression Model



Residual Analysis



Address Heteroscedasticity



Cross-Validation



Evaluate on Test Data



Final Robust Model

Useful Evaluation Metrics

For house-price prediction, use metrics such as:

- MAE – Mean Absolute Error

- MSE – Mean Squared Error
- RMSE – Root Mean Squared Error
- (R^2) – Coefficient of Determination

Example:

```
from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("MAE:", mae)

print("RMSE:", rmse)

print("R2 Score:", r2)
```

Final Answer for Question 10

To build a robust real-estate regression model, I would first perform **EDA and data cleaning**, then split the data into training and testing sets. I would diagnose heteroscedasticity using residual plots and, if necessary, use transformations such as a log transformation, robust standard errors, or weighted regression. For multicollinearity, I would examine the correlation matrix and VIF, remove or combine redundant predictors when justified, or use regularization such as Ridge Regression. Categorical variables such as location would be encoded appropriately. Finally, I would use cross-validation and evaluate the final model on unseen test data using MAE, RMSE, and (R^2).

Quick Revision

Topic	Key Point
Simple Linear Regression	One X predicts one Y
Multiple Linear Regression	Multiple X variables predict Y
Polynomial Regression	Uses powers of X to model curvature
Slope	Change in predicted Y for one-unit X increase
Intercept	Predicted Y when X = 0
Residual	Actual – Predicted
Heteroscedasticity	Non-constant residual variance
Homoscedasticity	Constant residual variance
Multicollinearity	Predictors are strongly related
VIF	Diagnostic for multicollinearity

Ridge	L2 regularization
Lasso	L1 regularization
MAE	Average absolute error
RMSE	Square root of average squared error
R^2	Proportion of target variation explained by model

Key Formulas

Simple Linear Regression

$$\hat{Y} = \beta_0 + \beta_1 X$$

Multiple Linear Regression

$$\hat{Y} = \beta_0 +$$

$$\beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k$$

Residual

$$e = Y - \hat{Y}$$

Polynomial Regression

$$\hat{Y} = \beta_0 + \beta_1$$

$X +$

β

2

X

2

$+ \dots +$

β

d

X

d

Easy Memory Trick

**Simple $\rightarrow$ One X Multiple $\rightarrow$ Many X's Polynomial $\rightarrow$ X, X^2 , X^3 ... Residual $\rightarrow$ Actual
– Predicted Heteroscedasticity $\rightarrow$ Unequal error spread Multicollinearity $\rightarrow$ X
variables are highly related**
